
SOLARFREN'S COLLECTION

INFOGRAPHIC MEGA LIBRARY

BEGINNER HANDBOOK

From your first folder to your first tested project.

Game systems · AI prompts · Rust · Modding

Research · Databases · Safe publishing



WINDOWS · macOS · LINUX

Organized explanations, practical exercises, and reusable prompts.

Full-size A4 edition · 3 October 2026

Contents

Click a chapter or section to jump to it. Page numbers match the printed footer.
PDF bookmarks provide the same navigation.

01 · Start here	6
What this handbook gives you	6
Choose your reading path	6
How to use this PDF and the library	7
Your learning milestones	7
02 · Computer basics on your operating system	8
The few words you need immediately	8
Find your OS and open your files	8
Use the browser and save a download	9
Copy, paste, save, and undo	9
03 · Files, project folders, and backups	10
What a filename tells you	10
Make a practice project folder	10
Paths: addresses for files	11
Downloads, ZIP files, and installers	11
Keep a recoverable copy	11
04 · Your first terminal commands	12
What the terminal actually does	12
Find your location before changing anything	12
A small folder exercise	12
Pasting, errors, and stopping a command	13
05 · Choose the right way to build	14
Start with an outcome, not a language	14
A practical decision sequence	14
Host, donor, and vertical slice	15
Tool language follows the host	15
06 · AI tools: desktop, browser, and coding agents	16
Understand the interfaces	16
Start with the browser or official desktop download	16
Terminal coding tools are optional	16
A useful first request	17
07 · Run your first playable practice project	19
A working exercise on Windows, macOS, or Linux	19

Try the stamina mechanic	19
Try movement and blocks	19
Write the rules you observed	20
Make one change at a time	20
08 · What Rust is, and your first Rust program	21
Rust in ordinary language	21
Install only if you chose the Rust path	21
Create and run a project	22
Run the meaningful checks	23
Modules, crates, and builds	23
09 · How reusable game systems fit together	24
The main pieces of a game	24
Agree on units and responsibilities	24
Timing changes the result	25
Why a shared interface helps	25
Save data instead of hardcoding every value	25
10 · The database method, without the mystery	26
Keep findings somewhere better than a long chat	26
A complete JSON example	27
Try SQLite in your browser	27
Scale the recordkeeping carefully	28
11 · Research and reverse engineering	29
Start with observable behavior	29
Match the tool to the format	29
A small Ghidra project	30
Evidence, guesses, and verification	30
Decompilation communities and debug builds	31
12 · Ghidra, Gemini, and MCP	32
What the extra connection adds	32
Prerequisites before you connect anything	32
The connection sequence	32
Keep a local bridge local	33
A focused analysis prompt	33
13 · Passthrough: connect two running worlds	34
The concept behind the posters	34
First connect state, then consider images	34
Why two images are harder than one	34
Collision and interaction need real rules	35
Stage the bridge project	35

The platform and online boundary	36
14 · Rewrites: build your own runtime	37
What a rewrite changes	37
Define which kind of fidelity you need	37
A start-to-finish small rewrite plan	37
AI assistance that produces reviewable work	38
Scale after the slice works	38
15 · Turn the named game ideas into small projects	39
BONELAB and Project Third Eye-style tools	39
Elden Ring-inspired combat plus Minecraft-inspired building	39
Minecraft inside GTA V	40
Garry's Mod plus voxel building	40
Postal 2 inside Minecraft	40
MW2, Skate 3, Sonic, and GoldenEye examples	41
A project card for any of these ideas	41
16 · Mega mashups and combination math	42
Keep the imagination, shrink the first build	42
There are two different counting questions	43
Correct examples from the library's notes	43
Use constraints to make design manageable	44
17 · Test, debug, and document the result	45
Three different kinds of proof	45
Test the boundary, not just the happy path	45
Debug one cause at a time	45
Write a field note	46
Package a result someone can use	46
18 · GitHub, public files, and GitHub Pages	47
What Git and GitHub each do	47
A beginner publication sequence	47
Keep secrets and personal archives out	48
Websites cannot hide a browser-side secret	48
Sharing an infographic or the handbook	48
19 · Subscriptions, student offers, and payment claims	49
Learn first; buy for a specific need	49
The four-month student offer	49
Pay-in-four and Google Play: what the image proposes	49
A smaller first payment does not reduce the total	50
20 · Troubleshooting without making the problem bigger	51
Capture useful context	51

OS-specific reminders	52
PATH, ports, and versions in plain language	52
A short request for debugging help	53
21 · Copy-paste prompts and a practical next step	54
Use the image as a frame of reference	54
Prompt A: route selection for a complete beginner	54
Prompt B: build one original mechanic	54
Prompt C: investigate a passthrough	54
Prompt D: BONELAB-native first slice	55
Prompt E: organize research and continue later	55
A sequence you can finish	55
A · Plain-language glossary	56
B · Coverage of the original collection	60
What was included and consolidated	60
Supporting discussion and exclusions	62
Corrections and limitations that matter	63
C · Sources, verification, and exercise files	64
How the sources are used	64
Companion files	66

01 · Start here

What this handbook gives you

This is SolarFren's plain-language companion to the Infographic Mega Library. It brings the useful ideas from 32 infographics, supporting references, and the original discussion into one learning path. Repeated Rust explanations and game-fusion pipelines appear once, followed by examples that refer back to them.

You do not need programming experience to begin. Start with the computer basics, run the small practice project, and choose one project route. The later research chapters are there when you need them. You can finish the first exercise without buying an AI subscription, owning a particular commercial game, or installing developer tools.

The owner made these AI-generated images mainly as one-shot brainstorming, frame-of-reference material, and copy-paste starting prompts for other AI assistants. They were not intended to be 100% accurate technical manuals. An illustration is not proof of a working mod. This handbook preserves that creative intent, adds original teaching examples, verifies selected practical instructions against primary documentation, and labels the ambitious ideas as concepts. Exact tool versions and offers can change after the verification date: 3 October 2026.

What “complete” means here: all retained subject areas are explained and indexed. It does not mean that the imaginary mashups have been implemented. Three images were missing from the original export; their contents cannot be reconstructed reliably.

Choose your reading path

Your starting point	Read first	Your first result
First time using a computer	Chapters 2-4, then 7	Open files, make a folder, run a practice game
Want to add something to a game	Chapters 5, 15, 17	A small mod plan matched to your actual game
Want to understand all the Rust pictures	Chapters 8-10	A working Rust program and a mechanic record
Want to research game internals	Chapters 10-12	A documented observation and a small analysis project
Want a live cross-game bridge	Chapters 5, 13, 17	A staged bridge plan with measurable checkpoints

Your starting point	Read first	Your first result
Want your own original game	Chapters 7–10, 14, 16	A tiny prototype with rules you can test

Do not install every tool in this book. A browser exercise needs a browser. A Minecraft Java mod needs the matching Java/modding toolchain. A Rust rewrite needs Rust. A BONELAB content project commonly starts with its Unity/Marrow tools. The chosen route determines the tools.

How to use this PDF and the library

Use the table of contents or the PDF reader's bookmarks panel to jump between chapters. Links printed in teal are clickable. Search the PDF with Ctrl+F on Windows/Linux or Command+F on macOS. Increase the PDF zoom until the body text feels comfortable; the pages use selectable text, not pictures of text.

The [online library](#) preserves the original images. Search a title, choose a topic, and open an item. **Fit** shows the whole infographic, **100%** shows its original pixels, and **+ / –** changes magnification. Scroll or drag to read different parts. Use **Share page** to copy a link that has its own image preview. Social platforms decide when to display or refresh a preview.

The source coverage appendix maps each retained infographic to this handbook. Spam and exact repeated uploads stay in separate archive collections and are not repeated as instructional material.

Your learning milestones

1. Find and save a file, make a practice folder, and keep a backup.
2. Open the Mechanics Playground and test its rules.
3. Describe one mechanic in ordinary words and record its parameters.
4. Pick a native mod, bridge, or rewrite route for one small goal.
5. Build or request one working feature, test it, and write down the result.
6. Package your own work with clear instructions and publish only intended public files.

Checkpoint: you know which chapters you need first. It is fine to stop after the browser exercise and return later.

02 · Computer basics on your operating system

The few words you need immediately

A **computer** runs programs. Its **operating system**, or OS, manages the screen, storage, and hardware. Windows, macOS, and Linux are different operating systems. An **application**, or app, is a program you open to do something: browse websites, edit text, or play a game.

A **window** is an app's rectangle on the screen. A **browser tab** is a page inside a browser window. A **file** holds information; a **folder** holds files and other folders. The **desktop** is the main screen behind your app windows. It is also often a folder where people save shortcuts and files.

Click means press the primary mouse button once. **Double-click** means press it twice quickly. **Right-click** opens a menu; on a trackpad, a two-finger click often does the same thing. **Drag** means hold the button while moving the pointer. **Scroll** moves through content with a wheel, trackpad, or scroll bar.

Find your OS and open your files

Task	Windows	macOS	Linux desktop
Find system details	Start → Settings → System → About	Apple menu → About This Mac	Settings → About or System Information
Open your files	Press Windows+E for File Explorer	Click Finder in the Dock	Open Files, Dolphin, Thunar, or your file manager
Find a download	Select Downloads in the file manager	Select Downloads in Finder	Select Downloads in the file manager
Start an installed app	Start menu, then type its name	Spotlight: Command+Space, then type its name	Application menu, then search its name
Switch between open apps	Alt+Tab	Command+Tab	Usually Alt+Tab

Linux comes in distributions such as Ubuntu, Debian, Fedora, and Arch. Desktop layouts differ. If a menu label differs, search for the task or app rather than assuming your computer is broken. Windows File Explorer's official help explains its file and folder controls. [\[S01\]](#)

Use the browser and save a download

1. Open your browser: for example Edge, Firefox, Chrome, or Safari.
2. Click the address bar at the top. Type a full website address and press Enter/Return. Search results and advertisements are different from the actual address you entered.
3. Open a link with one click. To keep your current page, right-click the link and choose the option to open it in a new tab.
4. Click a site's download link. If asked where to save it, choose Downloads first. Wait until the browser says the download is complete.
5. Open Downloads in your file manager. Check the filename and type before opening it. A PDF is a document; an installer is a program that changes what is installed on your computer.

You can usually open this PDF in your browser or your OS's document viewer. The download arrow in the PDF toolbar saves a local copy. When printing, choose your paper size and “Fit” or “Fit to printable area” if the printer needs it. Color is helpful but the explanations also work in grayscale.

Copy, paste, save, and undo

Action	Windows / Linux	macOS
Copy selected text	Ctrl+C	Command+C
Paste copied text	Ctrl+V	Command+V
Save a document	Ctrl+S	Command+S
Undo the last edit	Ctrl+Z	Command+Z
Select all text in the active field	Ctrl+A	Command+A
Find text	Ctrl+F	Command+F

Click in the place where you want the pasted text before using Paste. These shortcuts describe ordinary apps; terminals have a few differences explained in chapter 4. Keep a simple text note for your project. On Windows use Notepad; on macOS use TextEdit in plain-text mode; on Linux use the desktop's text editor. A word processor may add formatting that is unsuitable for code.

Checkpoint: open Downloads, locate this PDF, and copy one sentence into a saved text note. No terminal is needed yet.

03 · Files, project folders, and backups

What a filename tells you

The last part of a filename is its **extension**. Common examples are `.pdf` for a document, `.png` or `.jpg` for an image, `.txt` for plain text, `.html` for a web page, and `.zip` for a compressed collection. `.rs` is Rust source code; `.java` is Java source code; `.cs` is C# source code. These are instructions for a toolchain, not interchangeable “game files.”

Show extensions before editing code. On Windows 11, use File Explorer → View → Show → File name extensions. Other versions have a similar option in View or folder settings. On macOS, Finder → Settings → Advanced can show all filename extensions. Linux file managers usually display the full name; inspect Properties if needed. Renaming `notes.txt` to `notes.png` does not turn text into an image. [\[S02\]](#)

Make a practice project folder

1. Open your Documents folder, or another folder where you normally keep your own work.
2. Right-click an empty area and choose New Folder. Name it `SolarFren-Projects`.
3. Open it and create another folder named `first-prototype`.
4. Save a plain-text file named `notes.txt` inside `first-prototype`.
5. Write your goal in one sentence. Example: “I want to understand a roll that uses stamina and blocks that stop movement.”

This folder belongs to you and avoids editing inside a game's installation. On a shared or managed computer, choose a personal folder where your account can save files.

```
SolarFren-Projects/  
  first-prototype/  
    notes.txt  
    backups/  
    research/  
    prototype/  
    tests/  
    release/
```

You can add the subfolders when you need them. `research` is for observations, `prototype` for your editable project, `tests` for repeatable checks, and `release` for the files intended for other people. The exercise works even if you create only the first two folders.

Paths: addresses for files

A **path** tells a program where a file is. Windows often writes `C:\Users\YourName\Documents\first-prototype`. macOS commonly uses `/Users/YourName/Documents/first-prototype`. Linux commonly uses `/home/yourname/Documents/first-prototype`. These are examples: use your actual account name and folder.

An **absolute path** starts from a drive or filesystem root. A **relative path** starts from the folder the program is currently using. `notes.txt` means the note in the current folder. `research/notes.txt` means the note inside its research subfolder.

Put paths containing spaces in quotation marks when typing terminal commands. Do not type placeholder text such as `<your folder>` literally. Angle brackets in a guide often mean “replace this with your value.”

Downloads, ZIP files, and installers

A ZIP is a box of files. Download it, then extract it before using a project inside it. Windows offers **Extract All**; macOS normally extracts a ZIP when you double-click it; Linux offers **Extract** in the archive app or right-click menu. Extraction makes an ordinary folder. Opening a file while it is still inside an archive can hide related files from the program.

An installer is different. Windows often uses `.exe` or `.msi`; macOS often uses `.dmg` or `.pkg`; Linux may use a distribution package, app store, repository, or `AppImage`. Follow the software publisher's instructions for your OS and CPU architecture. “x64/amd64” and “arm64” are different builds; they do not mean different subscription plans.

Keep a recoverable copy

Before changing a project, close it and copy the project folder to backups or a separate storage location. Give the copy a clear date, such as `prototype-before-roll-2026-10-03`. A backup on the same disk protects against editing mistakes; a separate disk or trusted backup service also helps if that disk fails.

For a game mod, locate and back up the relevant saves using the game's instructions. A cloud-synced save can sync a mistake, so it is not automatically an independent backup. Use a disposable test world or profile for early experiments.

Git, explained in chapter 18, records project checkpoints. It complements backups. It should not contain private credentials or copied commercial installations.

Checkpoint: find your `notes.txt` again after closing the editor, and make one dated backup copy. If you can do that, you can safely organize a small project.

04 · Your first terminal commands

What the terminal actually does

A **terminal** is a window where you type commands. The **shell** interprets them. PowerShell, Bash, and Zsh are shells. The terminal is another way to work with files and tools; it does not automatically make a task advanced or dangerous.

A command normally has a program name followed by arguments. In `cargo new roll-demo`, Cargo is the program, `new` is the requested action, and `roll-demo` is the project name. A **prompt**, such as `$` or `PS C:\...>`, is the shell's indicator that it is ready. Do not copy the prompt symbol into a command.

Open the terminal in your practice folder where possible. On Windows, search Start for PowerShell or use File Explorer's **Open in Terminal** option. On macOS, open Terminal from Applications → Utilities or Spotlight. On Linux, search the application menu for Terminal; your file manager may offer **Open Terminal Here**. Apple's Terminal guide explains opening and quitting the app. [\[S03\]](#)

Find your location before changing anything

In Windows PowerShell:

```
Get-Location  
Get-ChildItem
```

In macOS Terminal or a Linux Bash/Zsh terminal:

```
pwd  
ls
```

The first command prints the current folder. The second lists what is inside it. They do not delete anything. If the folder is not your practice folder, either reopen the terminal there or use `cd` followed by its real path in quotes.

```
cd "the actual path to your practice folder"
```

That line is an example shape, not a command to paste unchanged. `cd` means change directory. `cd ..` moves to the parent folder. A command runs in the current folder unless it explicitly uses another path.

A small folder exercise

After confirming that you are in `first-prototype`, run these two commands one at a time. They work in PowerShell, Bash, and Zsh:

```
mkdir terminal-practice  
cd terminal-practice
```

Now check your location again. Open the file manager and confirm that `terminal-practice` exists inside your project. Run `cd ..` to return to the project

folder. An error saying the folder already exists simply means it was created before; check where it is rather than creating copies with random names.

Pasting, errors, and stopping a command

Windows Terminal usually supports Ctrl+V to paste. Linux terminals commonly use Ctrl+Shift+V. macOS Terminal uses Command+V. You can also use a terminal's Edit or right-click Paste command. In terminals, Ctrl+C generally interrupts a running command instead of copying text; select text and use the app's Copy menu when unsure.

Run one command, wait for the prompt to return, then read its output. If it fails, copy the first meaningful error and the command that produced it into your notes. A later error may only be a consequence of the first failure.

`sudo` on Linux/macOS asks to act with elevated privileges. Administrator terminals on Windows do something similar. Ordinary project creation should not need those privileges. An installer may legitimately need them; understand that it is installing software before granting them. Password typing at a terminal may show no dots or letters.

Checkpoint: you can identify the current folder, list files, and create a subfolder. Leave deletion, permissions changes, and operating-system repair commands until you understand their effect.

05 · Choose the right way to build

Start with an outcome, not a language

“Combine games” can mean several different things. Do you want a new item inside an existing game, the visual appearance of one character, a live connection between two running games, or your own independent game? These outcomes require different work.

Choose the smallest route that can deliver the feature you want. An existing modding SDK may already provide movement, input, physics, rendering, and saving. Rewriting all of those systems before adding one tool makes the project much larger.

Route	What runs	What you create	Main difficulty
Native mod or content recreation	The original host game	Its supported mod/content package	Learning its API and packaging rules
Asset or content port	The host game	Converted permitted content and host behavior	Formats, rigs, scale, dependencies, rights
Native bindings / FFI	A program plus compatible libraries	A boundary between languages	Matching types, lifetimes, and binary interfaces
Passthrough bridge	Two games or processes	A mod for each side plus communication	Synchronizing state, timing, and sometimes images
Rewrite / original prototype	Your new program	New implementations of selected rules	Rebuilding and integrating enough systems

FFI means foreign function interface: calling code across a language boundary. It works when there is a callable compatible interface. It does not make an arbitrary game executable into a reusable library.

A practical decision sequence

1. Write one observable goal: “A car can be spawned, entered, and driven in my BONELAB test level.”

2. Check whether the host's supported tools can express it. If yes, start with a native mod or recreation.
3. If appearance matters, decide whether you can make original art or have permission to convert the required assets.
4. If an essential live behavior really depends on a second process, investigate a bridge as a separate engineering project.
5. If you need a standalone runtime or control over every rule, build an original prototype or rewrite selected mechanics.

This is a decision sequence, not a promise that every route is available for every game. Engine version, operating system, hardware, licensing, tool support, and online restrictions all matter.

Host, donor, and vertical slice

The **host** is the game or app where the user experiences the feature. The **donor** is a source of permitted content or inspiration. In a passthrough project the donor may remain running; in a recreation it usually does not.

A **vertical slice** is one small feature that works all the way through. “Press a key, spend stamina, show the result” is a slice. A folder full of incomplete movement, combat, terrain, inventory, and multiplayer code is not one.

Make the first slice small enough to describe in a short checklist. For a cross-game experiment, one marker that follows another process's position is a better first milestone than a shared open world.

Tool language follows the host

Minecraft Java mod development commonly uses Java and a selected loader. Garry's Mod scripting uses Lua. BONELAB's Marrow content workflow uses Unity tools; custom behavior depends on what the supported SDK and selected mod route expose. Rust is valuable for its own native tools or runtime, and can also participate in interfaces, but it is not a prerequisite for every mod. See chapter 15 for the example routes. [\[S14\]](#) [\[S15\]](#) [\[S16\]](#)

Checkpoint: write the chosen route, why it fits, and the one thing you will test first. If you cannot choose yet, use the browser exercise in chapter 7 while gathering the game's exact details.

06 · AI tools: desktop, browser, and coding agents

Understand the interfaces

A browser chat is useful for explanations, planning, and reviewing short examples. A desktop chat app gives another interface to an AI service. A coding agent can inspect a project and use tools when the environment and permissions allow it. A chat answer alone does not mean that files were actually created, a compiler ran, or a game was tested.

The AI needs the relevant context: your goal, OS, exact game/tool versions, project folder, chosen route, and the first feature. It does not need your whole home folder or account secrets. Ask for explanations you can follow and measurable results you can verify.

Start with the browser or official desktop download

For browser use, open the official service website, sign in, and try an explanatory request before any automation. For desktop use, start from the publisher's download page, choose your OS and architecture, install using the displayed instructions, and open it from your normal app menu.

OpenAI's current quickstart lists a ChatGPT desktop app for macOS, Windows, and Linux. Linux distribution/package instructions are on its Linux app page. Anthropic's Claude download page currently lists a Linux beta for Ubuntu/Debian on x64 and arm64, alongside Windows and macOS downloads. Check the current requirements for your machine rather than assuming every Linux distribution uses the same package. [\[S04\]](#) [\[S05\]](#) [\[S06\]](#)

An app that opens successfully and accepts a normal chat message is a useful first checkpoint. Installation is not complete merely because a download exists. Some account features require an eligible plan or workspace; this handbook does not require you to buy one to begin.

Terminal coding tools are optional

Only install a CLI after you are comfortable with chapter 4. Use the official installation page for your chosen tool and copy the command for your actual shell. Do not combine Windows PowerShell, Windows CMD, and Bash installation snippets.

Tool	What it is	Start after installation	Check
Codex CLI	OpenAI's terminal coding agent	codex in your project folder	codex --version
Claude Code	Anthropic's coding agent	claude in your project folder	claude --version
Gemini CLI	Google's terminal AI tool	gemini in your project folder	gemini --version

Codex's official CLI page provides installation choices and first-run sign-in. Start it in a project folder and choose an offered authentication method. Review permissions before tasks that change files or run commands. The archived infographic's --auto-edit, --login, and --upgrade snippets are not this handbook's installation procedure; use the current CLI help and documentation. [S07]

Claude Code's official quickstart has separate installers for Bash and PowerShell, as well as package-manager choices. Open a fresh terminal after installation, check its version, and run claude to complete the login flow. It is distinct from the Claude desktop application's launch command. [S08]

Gemini CLI's installation page currently requires Node.js 20 or later and lists supported/recommended OS versions. A current supported Node.js LTS release from the official download page is the sensible starting point. If Node/npm are installed, check them, then install the official package and start it: [S09] [S10]

```
node --version
npm --version
npm install -g @google/gemini-cli
gemini --version
gemini
```

The npm installation contacts the package registry and installs software; it is not a read-only version check. If it reports a permission problem, follow Node's or Gemini's installation guidance for your setup. Do not immediately rerun an unknown package command as administrator.

A useful first request

I am a first-time learner using [my OS].
My project folder is [my actual folder].
Explain the files already here and the smallest next step.
Use plain language. Explain a command before asking me to run it.
Show the expected result and stop at the first unresolved error.
Do not upload, delete, or install anything as part of this request.

Replace the two bracketed fields. This first request gives you a readable inventory. For a later coding request, explicitly ask for the file changes and local checks you actually want.

Use one capable assistant and one small task first. Multiple models are optional. They do not automatically agree, produce correct code, or overcome missing tools. Ask an assistant to distinguish **observed**, **inferred**, and **untested** claims.

Checkpoint: the chosen interface works, and you can describe what it can access. If you only want explanations, browser chat is enough.

07 · Run your first playable practice project

A working exercise on Windows, macOS, or Linux

Open the [Mechanics Playground](#). It is a small original browser exercise made for this handbook. It has a stamina counter, a roll action, a movable player marker, and a grid where you place or remove blocks. It uses no commercial game files and requires no account.

To keep an offline copy, right-click the linked page and use **Save link as** or open it and use **Save Page As**. Save `mechanics-playground.html` into your practice folder. It is one self-contained HTML file. Double-click that file to open it with your browser, or use the browser's Open File command. If a browser saves a complete-page folder as well, the exercise still needs only the HTML file's embedded code.

This illustrates ideas found in the Elden Ring/Minecraft starter infographics. Its numbers are invented teaching values. It does not reproduce either game's combat or terrain engine, and its roll button does not animate or move the marker.

Try the stamina mechanic

1. Find the stamina display. It starts at **100 / 100**.
2. Press **Roll** once. The value becomes **80 / 100** because one accepted roll costs 20.
3. Press Roll four more times. Five rolls total reduce it to zero.
4. Press it a sixth time. The action is refused and the message tells you to recover.
5. Press **Recover** twice. Each press adds 10, giving enough stamina for one more roll.
6. Keep pressing Recover. The display stops at 100, which is the maximum.

You have tested a condition, a resource cost, and a limit. These are small pieces of game logic. The same structure can describe firing a weapon with ammunition or purchasing an item with coins.

Try movement and blocks

The player is the yellow outlined square with a dot. The grid has eight rows and eight columns. Click an empty square next to the player to place a green block. Use the direction buttons to try moving onto that block. The move is refused. Click the green block again to remove it, then try the same move. It now succeeds.

Try placing a block on the player's current square. The exercise refuses it. Try moving past an outer edge of the grid. That move is also refused. Press **Reset everything** to restore the start.

The grid uses coordinates: a column says how far across, and a row says how far down. The displayed coordinates start at 1. A program may count from 0 internally, so the first column can have internal index 0 and displayed label 1.

Write the rules you observed

Create a short entry in `notes.txt`:

Roll: accepted when stamina is at least 20.
Accepted roll: subtract exactly 20.
Recovery: add 10, but never exceed 100.
Movement: stay in the grid and avoid blocks.
Block toggle: add/remove a block, except on the player.
Reset: return to the initial state.

This is already a **behavior specification**: a description of what the program should do. You do not need decompilation to describe this exercise. Later you can turn these rules into structured data or code.

Make one change at a time

If you want a first coding task, make a backup of the HTML file and ask an assistant to change only the roll cost from 20 to 25. It should update the behavior, button text, explanations, and checks together. Then reload the changed file. Four rolls should reach zero; the fifth should be refused. Restore the backup if the behavior is wrong.

For an editable engine project later, Godot's official learning path covers scenes, nodes, scripting, and a first 2D game. Its full first-game tutorial assumes the earlier basics, so read those first if code is unfamiliar. You can make original 2D experiments before taking on 3D worlds or a Rust engine. [\[S11\]](#)

Checkpoint: you can explain the exercise's rules and demonstrate both an accepted action and a refused action. That is a finished first prototype milestone.

08 · What Rust is, and your first Rust program

Rust in ordinary language

Rust is a programming language. You write source code in text files, and its compiler turns that code into a program for a target platform. Rust is useful for native tools, file parsers, servers, and performance-sensitive systems. Cargo manages a Rust project's build and dependencies. Rustup installs and manages the Rust toolchain.

The infographic titles “Why Rust Feels Supernatural,” “Why Rust Dominates,” and “Why Rust Is Perfect” express enthusiasm. The useful point is that Rust can provide clear types, modules, and interfaces for a collection of systems. It does not recover a game's source automatically, verify an AI answer, or make every project cross-platform without additional work.

Ownership describes responsibility for a value and when it is released.

Borrowing lets code use a value without taking ownership. The compiler checks many rules before the program runs. In ordinary safe Rust, these rules help prevent common memory-access bugs. Rust can still have logic bugs, leaks, unsafe-code mistakes, dependency vulnerabilities, and incorrect interfaces to other languages. [\[S12\]](#)

Install only if you chose the Rust path

Open the official Rust installation instructions. On Windows, use the official rustup installer and follow its prompts for the required Microsoft C++ build tools. On macOS, follow the Rust instructions for the compiler/linker tools; you may need Apple's command line tools. On Linux, use the toolchain prerequisites appropriate to your distribution. The official Rust Book documents the OS differences. [\[S13\]](#)

For macOS/Linux, the official Rust site provides a rustup shell installation command. Read it on that site before running it. This handbook deliberately avoids treating an archived screenshot of an installer as a permanent source of current commands.

Close and reopen the terminal after installation. Run:

```
rustc --version
cargo --version
```

Each should print a version. If it says the command cannot be found, see chapter 20. Stop here until both work. You need a working compiler and linker, not just files sitting in Downloads.

Create and run a project

Open a terminal in your practice parent folder, then run one command at a time:

```
cargo new roll-demo
cd roll-demo
cargo run
```

Cargo creates `Cargo.toml` and `src/main.rs`. The TOML file describes the project; `src/main.rs` contains the program. The starter should print a greeting. These commands work the same way once Cargo is installed on the three desktop OS families. [\[S17\]](#)

Open `src/main.rs` in a plain-text or code editor. Replace its contents with this complete teaching program, then save it:

```
fn try_roll(stamina: &mut u32, cost: u32) -> bool {
    if *stamina < cost {
        return false;
    }
    *stamina -= cost;
    true
}

fn main() {
    let mut stamina = 100;
    for attempt in 1..=6 {
        let accepted = try_roll(&mut stamina, 20);
        println!("Attempt {attempt}: {accepted}, left {stamina}");
    }
}

#[cfg(test)]
mod tests {
    use super::try_roll;

    #[test]
    fn exact_cost_succeeds_then_empty_is_refused() {
        let mut stamina = 20;
        assert!(try_roll(&mut stamina, 20));
        assert_eq!(stamina, 0);
        assert!(!try_roll(&mut stamina, 20));
        assert_eq!(stamina, 0);
    }
}
```

This is a console exercise, not a 3D game. `fn` starts a function. `let mut` creates a value that may change. `u32` is a nonnegative integer type. `&mut` passes permission to change an existing value, and `*stamina` accesses that value. `true` and `false` report whether a roll was accepted. The loop tries six rolls.

Run the meaningful checks

```
cargo run
cargo test
cargo fmt
cargo clippy
```

The run prints five accepted rolls with remaining values 80, 60, 40, 20, and 0, followed by a refused sixth roll at 0. The test checks the exact-cost boundary and confirms that refusing an empty roll does not change the value. Rust's built-in test support runs functions marked as tests; its book explains assertions and failures. [\[S18\]](#)

Formatting standardizes the code's layout. Clippy looks for additional problems or improvements. If those last two tools are absent, check your rustup components and follow the official toolchain documentation; the successful run and test are the first milestone.

Modules, crates, and builds

A **module** groups related code. A **crate** is a Rust compilation unit/package building block; downloadable crates are dependencies. Add dependencies when you need their functions, and keep the lockfile for a repeatable application build.

`cargo build --release` makes an optimized build for your current target. On Windows the executable commonly has `.exe`; on macOS/Linux it normally does not. A build for another OS may need that target's compiler/linker, libraries, SDK, and separate testing. Adding a Rust target does not by itself guarantee a working cross-platform game.

Use Rust when it solves your project's needs. A small Python automation script, Java mod, Lua addon, or Unity content package can be a better fit for another route.

Checkpoint: the program runs and its test passes. You understand that `.rs` is source code and Cargo builds it; a downloaded Rust file is not automatically a mod you can drop into every game.

09 · How reusable game systems fit together

The main pieces of a game

An **engine** provides reusable facilities such as a window, input, rendering, sound, and sometimes physics. A **game** combines those facilities with rules and content. A **runtime** is the running software and its environment. A **client** is the user-side program; a server, if there is one, handles selected shared services or state.

An **entity** is something the game tracks, such as a player, block, projectile, or car. A **component** is one kind of data attached to it, such as position, health, or velocity. A **system** applies rules to relevant data: movement updates positions, combat changes health, and recovery changes stamina.

ECS means entity-component-system. It is one architecture for organizing those parts. You can understand the idea without adopting an ECS library. Bevy is a Rust game engine that uses ECS; its APIs and compatible dependency versions change, so use documentation and examples for the release you actually select. [\[S19\]](#)

Agree on units and responsibilities

Two modules only combine cleanly when they agree on their contract. Decide what a position means, which axis points up, how big a unit is, who owns the player state, and which system is allowed to change it. A physics module using centimeters cannot silently exchange values with a renderer using meters.

Useful defaults for an original teaching prototype are seconds for elapsed time, meters for distance, and explicitly named units in notes. These are your own choices, not recovered constants from a commercial game.

System	Receives	Produces	Boundary to test
Input	Key/button state	A requested action	Held key versus one press
Movement	Action, position, elapsed time	Candidate next position	World edge and blocked cell
Stamina	Roll request, resource value	Accepted/refused action, new value	Exactly enough and too little
Block editing	Target cell and edit action	Updated occupied cells	Player cell and outside world

System	Receives	Produces	Boundary to test
Rendering	Current approved state	Screen image	Resize and readable UI
Saving	A complete state snapshot	A stored document	Save/reload preserves the state

Timing changes the result

Frame rate is how often a new image is shown. A **simulation step** updates the rules. They need not happen at the same rate. If movement adds a fixed distance on every rendered frame, a faster computer may move farther each second. Using elapsed seconds or a fixed simulation step avoids that particular mistake.

For reproducible physics, a fixed step is often useful, but “deterministic” is stronger than “fixed-step.” Floating-point behavior, random seeds, input order, threading, and external libraries can still affect reproducibility. Measure what your program actually guarantees.

Why a shared interface helps

An interface says what a module promises to do. For example, a world query may accept a location and return whether it is occupied. The movement system then asks the same question whether the underlying world uses a simple grid or a more advanced spatial structure.

An interface does not erase different assumptions. Skateboard movement, a walking character, and a car may need different acceleration rules, contact handling, or animation. Combine behavior through explicit adapters rather than assuming “physics” means the same thing everywhere.

Save data instead of hardcoding every value

Put adjustable values such as roll cost, recovery amount, and maximum stamina into a small configuration document once the prototype works. Validate the values when loading: a negative duration, missing field, or enormous world size should produce a useful error instead of strange behavior.

Serialization means turning structured values into a storage or transmission format such as JSON. **Deserialization** means reading them back. The format preserves data; it does not automatically implement the mechanic's code or prove that it is correct.

Checkpoint: sketch the handful of systems your first slice needs, their inputs/outputs, and one boundary case for each. Add rendering or physics complexity only when the existing slice works.

10 · The database method, without the mystery

Keep findings somewhere better than a long chat

The library's “database method” means saving research as structured, retrievable records. A database helps you answer focused questions such as “What evidence supports this roll cost?” or “Which modules depend on collision?” It does not automatically make the evidence correct.

Start with a text note when you have one mechanic. Add a spreadsheet, JSON, or SQLite when the number of findings makes searching and consistency difficult. They serve different needs: a spreadsheet is convenient for editing a small table, JSON is convenient for exchanging structured records, and SQLite supports queries and relationships in a local database file.

Record field	Example	Why it matters
Stable ID	demo.roll	Refers to the same mechanic after renaming
Source	Mechanics Playground, this edition	Says where the observation came from
Claim	Roll requires at least 20 stamina	States an actual rule
Status	Observed	Separates evidence from guesses
Parameters and units	cost 20 stamina points	Avoids ambiguous numbers
Evidence	Five accepted rolls; sixth refused	Makes the claim checkable
Dependencies	Stamina state and roll request	Shows what must exist first
Verification	Local exercise test passed	Records what was tested

For commercial-game research, include the exact game version, file hash if relevant, tool version, address/function reference, and the experiment or source supporting the claim. Do not label a guessed function name as recovered fact.

A complete JSON example

JSON is plain text with structured fields. Quoted words are strings; unquoted numbers are numeric values. This example describes our invented exercise, not Elden Ring's actual settings:

```
{
  "id": "demo.roll",
  "source": "Mechanics Playground",
  "status": "observed",
  "description": "Spend stamina to accept a roll request",
  "parameters": {
    "maximum_stamina": 100,
    "roll_cost": 20,
    "recovery_per_action": 10
  },
  "units": "stamina points",
  "dependencies": ["stamina", "roll_request"],
  "verification": "five rolls accepted; sixth refused"
}
```

Save it as `roll.json` if you want a data record. A JSON file does not run by itself. A program must read, validate, and use it. A missing comma, trailing comma, or unmatched quote can make JSON invalid.

Try SQLite in your browser

Open [SQLite Fiddle](#), the SQLite project's browser tool. Paste the following SQL into its SQL input area and use its Run/Execute control. SQL is the language used to ask a database to store and retrieve records. The same SQL can run in SQLite on Windows, macOS, or Linux. [\[S20\]](#)

```
CREATE TABLE IF NOT EXISTS mechanics (
  id TEXT PRIMARY KEY,
  description TEXT NOT NULL,
  cost INTEGER NOT NULL CHECK (cost >= 0),
  evidence TEXT NOT NULL
);

INSERT INTO mechanics (id, description, cost, evidence)
VALUES ('demo.roll', 'Spend stamina to roll', 20,
       'Five accepted; sixth refused')
ON CONFLICT(id) DO UPDATE SET
  description=excluded.description,
  cost=excluded.cost,
  evidence=excluded.evidence;

SELECT id, cost, evidence FROM mechanics;
```

Expected result: one row with `demo.roll`, cost 20, and the evidence text. Run the script again. It should still show one row rather than an accidental duplicate. This is an **idempotent** update: running it again leads to the same intended stored

result.

The browser exercise is for learning. Do not assume it has saved a durable file; use its available export feature or save your SQL separately. If you later install the SQLite CLI, opening `sqlite3 mechanics.db` creates or opens a local database file. End SQL statements with a semicolon; `.quit` leaves that CLI. [\[S21\]](#)

Scale the recordkeeping carefully

At larger scale, separate tables can describe files, functions, symbols, claims, evidence, dependencies, and tests. Link a claim to its source instead of repeating the full decompiled file inside every record. Index IDs and frequently queried fields. Keep the original research artifacts alongside the database where practical.

Parity checks compare what you expected to import with what was actually imported. Count files, compare hashes, report missing or truncated records, and check duplicate IDs. A database containing 1,000 rows is not complete merely because it is large.

The infographics show names such as `gamedb index`. Those are workflow/project-specific examples, not guaranteed commands installed with SQLite. Before using one, identify the actual repository, installation procedure, supported version, and command help. You can learn the method without that particular utility.

Checkpoint: you can query one mechanic and point to the evidence for it. You understand that a database organizes knowledge; the implementation and tests remain separate work.

11 · Research and reverse engineering

Start with observable behavior

Reverse engineering means studying a system to understand how it works. Begin with supported documentation, visible configuration, open source, and controlled observations when those answer the question. Inspecting binaries is useful when simpler evidence is insufficient, but it is a later skill.

Disassembly translates machine instructions into an assembly representation. **Decompilation** attempts to express compiled behavior as higher-level pseudocode or reconstructed source. The result often has guessed names and types, missing context, and compiler artifacts. It is not automatically the original source code or a buildable project.

Owning a game gives you a copy to use under its terms, not unrestricted rights to publish its files or recovered code. For this handbook's practice work, use your own program or material expressly allowed for learning. For a real project, check the applicable license and modding terms before distribution.

Match the tool to the format

Material	Typical research route	First question
Ordinary text scripts/configuration	Text editor and format documentation	Can I read the rule directly?
Native executable or library	Ghidra, IDA, or Binary Ninja	What architecture and binary format is it?
.NET assembly	A .NET-focused tool such as ILSpy	Is it actually a managed assembly?
Java classes / JAR	A Java bytecode tool such as Vineflower	Which Java/game version produced it?
Android package / DEX	Android-focused analysis such as JADX	What is code and what is an asset?
Unity IL2CPP build	Tools matched to its build and metadata	Are the required metadata files available?
Compiled scripts or packed data	A format-specific tool	Does this exact variant have supported tooling?

These are categories, not installation recipes. A `.dll` extension alone does not distinguish native code from `.NET`. A Unity project does not necessarily use the same build backend as another Unity project. Do not feed every file to the same decompiler and expect meaningful output.

A small Ghidra project

Download Ghidra from the official [NationalSecurityAgency/Ghidra](https://ghidra.sherlock-project.org/) releases. Extract the distribution into a new folder and read the `GettingStarted` document included with that release. Install the JDK it requires, then launch `ghidraRun.bat` on Windows or `ghidraRun` on macOS/Linux using its instructions. The current upstream guide describes Ghidra 12.2 with a 64-bit JDK 25 requirement; older images showing JDK 21 belong to another setup. [\[S22\]](#)

1. Use a tiny executable you compiled yourself, such as the chapter 8 exercise. For readable comparison, prefer your development build first.
2. In Ghidra, create a new non-shared project in your research folder.
3. Import the executable and review its detected format and architecture.
4. Open it in the analysis tool and allow its normal analysis to finish.
5. Explore functions and references. A stripped or optimized binary may have fewer useful names than the source you wrote.
6. Record one finding and compare it with your own source. Keep your conclusion modest if the evidence is incomplete.

The import is analysis; do not execute an unknown binary merely to inspect it. If Ghidra fails to start, first check the release's required Java version and its launch log. Fixing Java compatibility is different from changing the game.

Evidence, guesses, and verification

A **symbol** is a named program item. A **string** is text stored in the program. An **xref**, or cross-reference, links a use to another location. A **call graph** shows possible function-call relationships. These provide clues, but a string named “health” does not prove that every referencing function updates player health.

Record three layers: what you directly saw, what you infer from it, and how you would test that inference. A useful note is “This function compares a value with 20 before subtracting 20; in my compiled exercise that matches the roll rule.” A weaker note is “This must be Elden Ring's roll function” without any supporting context.

Decompilation communities and debug builds

The `decomp.dev` infographic points toward community reconstruction projects. Some **matching decompilation** projects aim to rebuild source that compiles to the same binary as a particular original build. That goal differs from a portable gameplay rewrite. Progress percentages, linking status, and licenses vary by project. This handbook could not retrieve `decomp.dev`'s live page during verification, so the poster's project percentages remain archived examples.

The Bully debug-build infographic illustrates why development artifacts can help research. Hidden Palace documents a Wii debug build with uncompiled ACT files, compiled CAT counterparts, loose DAT files, script debug information, and an in-game debug menu. Those can provide useful comparisons and labels. They do not supply every piece of the original engine or establish a redistribution license. [\[S23\]](#)

The GTA 6 “systems goldmine” poster is a brainstorming lens: traffic, missions, crowds, combat, inventory, and streaming can be studied as separate systems. It is not evidence that a complete GTA 6 decompilation or extraction pipeline is available. You can explore those general ideas with your own small traffic or mission simulation.

Checkpoint: one finding has a clear source, version, evidence, and uncertainty. Avoid mass extraction until you know what question the next artifact will answer.

12 · Ghidra, Gemini, and MCP

What the extra connection adds

MCP, the Model Context Protocol, is a standard that lets an AI application use external tools and data. An MCP server exposes capabilities; a compatible client connects to them. It does not automatically give an assistant every program on your computer or turn all tool output into verified facts. [\[S24\]](#)

A Ghidra bridge can let an assistant request function lists, decompilation, references, and other analysis from a loaded program. The specific bridge decides the available commands and permissions. Some bridges also modify names, types, comments, or run scripts. Treat those abilities as actual tool access, not merely a chat feature.

Prerequisites before you connect anything

You should already have a working Ghidra project from chapter 11 and a working Gemini CLI installation from chapter 6. Record the exact versions of Ghidra, Java, Python, Gemini, and the bridge. Take a copy of the Ghidra project before experiments that edit it.

The image names [GhidraPluginProject/ghidra-mcp](#). It is a third-party integration, not Ghidra's core distribution. Its README currently describes prerequisites aimed at Ghidra 12.0.3/JDK 21. That differs from the newer upstream Ghidra guide. Do not assume the newest Ghidra and an older bridge build are compatible. Follow a tested bridge release's requirements as a set. [\[S25\]](#)

The connection sequence

1. Read the bridge's current README and release notes. Confirm that they name a supported combination for your platform.
2. Install/build the bridge in a dedicated tool folder using that release's instructions. Check any script before granting installation privileges.
3. Enable its Ghidra extension as instructed, open your test program, and start the bridge service if required.
4. Register the bridge with Gemini using Gemini's current MCP configuration/help and the bridge's documented transport. A local Python process and an HTTP endpoint are not interchangeable configurations.
5. Confirm that Gemini lists the server and its tools. Request read-only program information first.
6. Ask it to explain one small function, save the result as a draft observation, and compare that explanation with Ghidra and your known source.

This is an intentionally version-aware setup sequence. The archived poster's setup script paths, port 8889, and endpoint names are specific examples; a different

bridge may not expose them. A tool list and a successful read-only call are the real connection checkpoint.

Keep a local bridge local

127.0.0.1 means the current computer's loopback interface. A service listening on 0.0.0.0 may accept connections through other interfaces. Keep experimental analysis services on loopback and leave router port forwarding out of the exercise. Do not expose a script-capable bridge to the internet just to make an AI connection work.

A localhost service is not a complete security boundary. Know what client can call it, what authentication or process permissions it has, and what it can change. If the assistant uses a cloud service, check which analysis text it sends there before analyzing sensitive material.

A focused analysis prompt

Use only read-only tools for this task.
Identify the loaded program and its architecture.
Choose one small function and explain its control flow.
For each conclusion, cite the address and supporting tool result.
Label uncertain names/types as guesses.
Do not rename, patch, execute scripts, or contact other services.
Save a short draft note with the evidence and next verification step.

Start with small, checkable questions. “Decompile everything” can consume time, produce incomplete records, and bury important uncertainties. Save verified findings in the structure from chapter 10 so another session does not repeat the same investigation.

Checkpoint: the connection can read one known program, and you can verify one explanation yourself. Disconnect the bridge when you no longer need it.

13 · Passthrough: connect two running worlds

The concept behind the posters

In a passthrough design, two processes remain alive. Mods or plugins provide controlled integration points, and a bridge exchanges state or events. A rendered overlay may make their output appear in one scene. That is different from creating a new engine that owns every rule.

The Minecraft-in-GTA-V and Minecraft-in-Elden-Ring posters describe possible camera, input, world, image, and event connections. Their diagrams are a frame of reference for an AI engineering discussion. They are not complete build instructions or proof that the pictured projects exist.

```
Host game + host mod
      |
  local bridge
state / events
      |
Companion game + companion mod

Optional separate layer: image/depth compositing
```

First connect state, then consider images

Choose one source of truth. For example, the host owns player position and the companion follows it. Send a simple message with a schema version, sequence number, timestamp, and named units. Confirm the receiver can display a marker at the right location.

Then test a single event, such as pressing a button to request an action. Give the event a unique ID so a repeated message does not apply damage or spawn an object twice. Define what happens when the companion is paused, disconnected, or slow.

IPC means inter-process communication. A local socket, pipe, WebSocket, or shared-memory segment can carry data between processes. Pick a mechanism both sides can support. Rust can implement a bridge if it is a good fit, but C++, Java, C#, or another supported language may be more direct for the selected game plugins.

Why two images are harder than one

Color frames are pictures. A **depth buffer** records distance information used by rendering. To place blocks convincingly between the host's buildings or characters, the compositor must compare compatible depth values, camera matrices, field of view, screen size, and coordinate systems.

Some renderers use reversed-Z or nonlinear depth values. Copying a grayscale-looking depth image from one game to another does not make the values comparable. The implementation must convert them correctly and account for near/far planes and projection. Missing depth access can limit the result to a simple overlay.

Reprojection adjusts a frame using a change in camera pose. It may reduce visible lag, but cannot recover every object hidden in the previous frame or eliminate all timing errors. First use test boxes, a fixed camera, and a scripted camera path before relying on complex live scenes.

Collision and interaction need real rules

Drawing a block does not make a car collide with it. Each simulation needs an appropriate collision representation. Proxy colliders, simplified terrain samples, and translated events can help, but they require explicit limits and update rules.

The posters propose mappings such as TNT to an explosion, arrows to impact events, or a Minecraft avatar to a host player. For each mapping, define the source event, destination effect, units, ownership, delay, and failure behavior. Do not read or write another process's arbitrary memory when a supported mod API provides the needed state.

Stage the bridge project

Stage	Build only this	Evidence before advancing
1. Reconnaissance	Version/tool/platform inventory	Both games work separately in test profiles
2. Fake endpoints	Two small test programs	Messages arrive and disconnect is handled
3. Game state	One host position and companion marker	Correct axes/scale and no feedback loop
4. One event	One action translated once	Duplicate IDs do not repeat its effect
5. Visual integration	One permitted model or simple overlay	Stable alignment during movement/resize
6. Depth/collision, if needed	A small controlled scene	Correct occlusion/contact under known cases

Stage	Build only this	Evidence before advancing
7. Package	Versioned files and instructions	Another test profile can install and remove it

These are engineering milestones, not guaranteed one-evening steps. Two games can require substantial RAM/GPU resources; test performance on the actual machine.

The platform and online boundary

A Windows-specific plugin, shared-memory API, or DirectX integration may make a bridge Windows-specific even when its companion Java app runs elsewhere. Compatibility layers are separate experiments, not a promise of support on every OS.

Use only the game's supported single-player mod arrangement for experiments. This handbook does not provide anti-cheat bypass instructions. If a planned hook requires defeating protection or breaks the game's rules, choose another permitted route. A backup and an offline test profile help preserve your normal saves and configuration.

Checkpoint: your first bridge goal is one state transfer or event with an explicit success check. You have not mistaken a composited picture for a shared simulation.

14 · Rewrites: build your own runtime

What a rewrite changes

A rewrite implements selected behaviors in a new codebase. Instead of synchronizing two live games, your program owns its input, world, rules, and presentation. This gives architectural freedom, but you must build the missing pieces yourself.

The “Rust Rewrite Client” prompt poster describes research → structured findings → new modules → testing. Treat it as a project outline. There is no verified one-click conversion from arbitrary installed games into a complete Rust source tree.

For an original prototype, you can work from observable design ideas without binary analysis. “A roll spends stamina” or “blocks occupy grid cells” is enough to start your own rules. If you are aiming for compatibility with a specific system, the evidence and verification requirements become much stricter.

Define which kind of fidelity you need

Inspired behavior: your own implementation follows a general design idea. You choose the numbers and may change them. **Behavioral compatibility:** your implementation aims to match selected inputs and outputs of a reference. **Binary matching:** a reconstruction aims to reproduce a specific compiled artifact. Those goals should not be mixed in a progress claim.

For a beginner, inspired behavior is the smallest useful path. Pick one familiar rule, original placeholder visuals, and an explicit test. You can later compare it with a reference where lawful and useful.

A start-to-finish small rewrite plan

- 1. Specify:** write the rule, parameters, units, boundary cases, and source status.
- 2. Implement the pure rule:** make a function that can be tested without a renderer. The chapter 8 stamina function is an example.
- 3. Present it:** show the result in a console or a basic window. Confirm the interface reflects the true state.
- 4. Add one second mechanic:** introduce grid movement or block editing, then define how it interacts with the first rule.
- 5. Store parameters:** validate configuration and report useful errors for invalid input.
- 6. Save/reload, if needed:** choose a documented format and confirm the loaded state matches what was saved.

7. Package and document: include controls, tested platforms, limitations, and a clean way to reset the prototype.

Use the smallest engine or framework that fits. A GUI engine may shorten iteration for visual content; a Rust engine may fit a systems-learning goal. A custom renderer should be a deliberate need, not an automatic prerequisite.

AI assistance that produces reviewable work

Give the assistant your mechanic specification and the exact tool/library version. Ask it to implement one module, explain the public interface, and run the relevant local checks. Require a clear distinction between compilation, automated tests, and actual gameplay testing.

An assistant should report unresolved errors and unavailable tests. If it cannot launch your VR game or test a macOS build, “not tested here” is the correct status. Generated pseudocode should not be labeled runnable code.

Scale after the slice works

Introduce separate modules for movement, stamina, inventory, terrain, combat, and saving as the project needs them. Keep dependencies clear: combat may consume stamina; rendering observes state; persistence stores an agreed snapshot. Avoid two modules silently changing the same values in incompatible ways.

Build and test for each target OS you intend to support. Native file paths, graphics libraries, plugins, and device APIs can differ. A Rust project is not “tested on every OS” because its source compiles on one machine.

Checkpoint: one original runtime implements and demonstrates a documented rule, with its known limitations written down. Expand one subsystem at a time.

15 · Turn the named game ideas into small projects

BONELAB and Project Third Eye-style tools

The chat asks how to put Project Third Eye-style VR tools and cars into BONELAB, and whether Rust files are necessary. Start by asking what behavior you want: one handheld tool, one drivable vehicle, and one test level. A visual resemblance, a native recreation, a content port, and a live passthrough are different goals.

BONELAB's official MarrowSDK documentation organizes content as pallets and crates, including levels, avatars, and spawnables. Begin with the SDK's current project setup and its supported Unity version. Do not assume arbitrary custom C# scripts are accepted by every content route; first establish what that SDK version and any required mod framework actually support. [\[S16\]](#)

For the first slice, use an original placeholder tool and a simple vehicle in a test level. Verify VR grabbing/input, object scale, collision, vehicle entry/exit, and reset. A car can require more behavior than a static spawnable, so identify the supported implementation route before building its art.

Rust is unnecessary unless a specific selected component benefits from it. Prefer Unity/Marrow-native recreation if it can deliver the behavior. Consider a separate runtime or bridge only when a required behavior cannot reasonably be implemented natively. Check BONELAB's platform and headset requirements separately from the OS used for authoring; a PDF that works on Linux does not make a particular VR game or mod tool supported there.

Elden Ring-inspired combat plus Minecraft-inspired building

The starter posters propose movement, a roll, stamina, and block place/break. Begin with the browser exercise, then an original 2D or simple 3D prototype. Use your own numbers, shapes, and names. Add a visible stamina bar and a grid; then test movement versus occupied cells.

For a later Minecraft Java mod, follow a loader's documentation for the exact game version. Fabric's official developer guide covers environment setup, project creation, launching a development game, and building a mod. A Bedrock addon uses a different route; do not assume a Java/Fabric .jar can be installed in Bedrock. [\[S14\]](#)

The Minecraft character/HUD inside Elden Ring posters describe a live bridge concept. Their first slice could be one permitted placeholder avatar and a simple HUD, followed by position synchronization. Rendering, input translation, depth, and collision are separate milestones. No completed Elden Ring bridge is supplied

by this handbook.

Minecraft inside GTA V

The archived diagram describes a Windows-oriented combination of a GTA plugin, Minecraft Fabric mod, local event channel, shared frames, and a compositor. It is a research proposal with specific version-sensitive assumptions. The indicated camera axes, scale, lighting, and depth calculations must be verified on the actual selected tools.

A useful first experiment is a standalone pair of mock programs exchanging position data, then a test scene with two differently colored boxes for depth alignment. That teaches the integration questions without requiring a full game hook. If you later work with GTA V, use a supported offline/single-player mod arrangement and its current modding documentation. Do not apply this experimental design to GTA Online.

Garry's Mod plus voxel building

The GMod/Minecraft poster mixes props, blocks, constraints, ragdolls, contraptions, survival, crafting, and redstone-like logic. Choose only one first: a small grid of original blocks that can be spawned and removed in a supported sandbox addon.

Garry's Mod's publisher-maintained wiki documents Lua scripting and game APIs. Use an introductory addon that matches the current game, then add one block action and test the collision. A redstone-style circuit simulator, a moving voxel vehicle, and multiplayer replication are later separate systems. Thousands of addons do not become mutually compatible merely because they all install. [\[S15\]](#)

Postal 2 inside Minecraft

The preserved community prompt proposes adapting SkyCraft so Minecraft becomes the host and Postal 2 becomes a companion. This is an initial request for an AI to investigate, not an established Postal 2 port. Begin by confirming the actual games, versions, available modding APIs, and required behavior.

SkyCraft's current repository describes Minecraft integration with **Skyrim**, using an SKSE plugin and Fabric mod. It lists Skyrim runtime requirements and a Windows-oriented launch path. Its current requirements differ from the old release link in the chat. That provides a concrete reference architecture, but does not establish Postal 2 compatibility. Adapting it may require major new host/companion integrations. [\[S26\]](#)

A simpler starting goal may be original Postal-style movement or a kick action recreated through Minecraft's supported mod APIs. Test one action, one target, and one effect. Add an NPC and a small event only after that works. Use original or permitted assets; do not package the donor installation.

If adapting a bridge remains the chosen route, first build a message test outside either game. The next milestone is one real state value or event passing through supported integration points. Keep all promised in-game behavior unverified until somebody tests it.

MW2, Skate 3, Sonic, and GoldenEye examples

The opening Rust infographic combines FPS weapon behavior, skateboard movement/tricks, and a voxel world. Treat those as three subsystem inspirations. Begin with one weapon rule and one movement rule inside an original test environment, rather than assuming the named community clients provide compatible reusable libraries.

The database-method image also mentions Sonic model/controller/animation work. Separate asset rigging and animation conversion from the controller's gameplay rules. A converted model does not bring its original movement code with it. GoldenEye appears as inspiration for FPS combat, mission objectives, and enemy behavior: one trigger that completes one objective is a manageable initial mission system.

The CHASM overview is an archived community snapshot about releases, playtests, clips, and showcases. Its invitation, project availability, and release details can change. Read each project's own current repository, licenses, release notes, and requirements before downloading or running it. A community announcement is evidence of an announcement, not a compatibility guarantee.

A project card for any of these ideas

Goal: one observable feature.

Host / inspiration source: exact names and editions.

OS and hardware: authoring machine and target runtime.

Versions: game, loader, engine, SDK, dependencies.

Route: native mod / content port / bridge / rewrite.

First slice: one input → one state change → one visible result.

Verification: precise steps and expected output.

Unknowns: unsupported tools, missing APIs, untested behavior.

Distribution: only my own or expressly permitted files.

Checkpoint: your exciting game idea now has a small implementation route and a test. You have not committed to a full engine rewrite before checking the host's available tools.

16 · Mega mashups and combination math

Keep the imagination, shrink the first build

“Any Game × Any Game,” “The Mega Game Combo Engine,” “Metal Gear Rising — Absurd Fusion,” and “What the Hell Is This Combo” are creative rosters. They propose combining open worlds, fast combat, vehicles, factories, physics chaos, survival, quests, horror, and multiplayer. The useful output is a menu of mechanics to explore.

Their “no limits” framing is imaginative, not a technical guarantee. Different engines, asset formats, animation rigs, world scales, hardware demands, and simulation rules create real constraints. Multiplayer also needs explicit authority, replication, validation, and a trust model.

Translate the posters into categories rather than a promise to copy every game:

Theme	Examples from the brainstorming roster	Small experiment
Movement and combat	Elden Ring, Wukong, Metal Gear Rising, Titanfall, DOOM	One dodge or jump with a resource/cooldown
City/world systems	GTA, RDR2, Cyberpunk, Cities: Skylines	Ten agents following a road graph
Voxels and construction	Minecraft, GMod, Teardown, Space Engineers	Place/remove blocks in a small area
Exploration and travel	No Man's Sky, Star Citizen, Outer Wilds, Kerbal	One vehicle visiting two locations
Survival and progression	Rust (the game), Subnautica, Project Zomboid	One resource, recipe, and need
Industry and automation	Factorio, Satisfactory	Two machines exchanging one item
Quests, simulation, and atmosphere	WoW, Baldur's Gate, The Sims, Silent Hill	One branching event with a saved outcome

Here **Rust the game** is separate from **Rust the programming language**. The posters also name many shooters, racing games, mining games, and RPGs. Those

names suggest mechanics to study; inclusion does not imply a runnable combined project or permission to redistribute their content.

There are two different counting questions

Choose distinct modules with no order. If you have N different modules and choose exactly k, the count is “N choose k,” written $C(N,k)$. Choosing combat, building, and driving is the same set regardless of the order you listed them.

Fill labeled slots with options. If five named slots each have ten choices, and every choice is independent, the count is $10 \times 10 \times 10 \times 10 \times 10 = \mathbf{100,000}$. Changing which option occupies a particular slot creates a different arrangement.

These counts describe a mathematical search space. They do not measure the number of fun, compatible, implemented games.

Correct examples from the library's notes

Calculation	Exact result	Meaning
Choose 2 from 30	435	Two distinct selected items
Choose 3 from 30	4,060	Three distinct selected items
Choose 4 from 30	27,405	Four distinct selected items
Choose 5 from 30	142,506	Five distinct selected items
Sum of choices of 2–5 from 30	174,406	All those subset sizes together
Sum of choices of 1–5 from 1,000	8,291,875,042,450	Roughly 8.29 trillion subsets
Five labeled slots, 100 choices each	10,000,000,000	Assumes independent slot choices

The reference note multiplies the large subset count by 100,000 configurations. That can be a useful toy assumption, but only if each subset actually has five independent configurable slots. Subsets containing one to four modules do not automatically have the same configuration count. State your assumptions before multiplying.

If each chosen module independently has ten configurations, count each subset size separately: $\sum C(N,k) \times 10^k$ over the sizes you allow. Compatibility rules can reduce the valid space sharply.

Use constraints to make design manageable

Set limits: one world representation, one physics authority, one player controller, one save format, and two or three mechanics. Reject combinations with incompatible assumptions unless you are deliberately building an adapter.

For a first fusion experiment, choose movement + stamina + block editing. After it works, add either combat or crafting, not every feature at once. Keep the mega roster as a future-ideas page. A completed small system teaches more than an enormous unverified generated project.

Checkpoint: you can tell a combination count from a working-build count, and have selected a small set with compatible responsibilities.

17 · Test, debug, and document the result

Three different kinds of proof

Build proof: the code compiles or the package is generated. **Automated behavior proof:** a repeatable check passes for a particular rule. **Runtime proof:** the actual app/game shows the intended behavior on a stated platform. One kind does not replace the other two.

A compilation success cannot prove that a VR grab feels correct. A gameplay clip cannot prove every save loads correctly. A useful release says which checks were performed and which were not.

Test the boundary, not just the happy path

Feature	Normal case	Boundary/failure case
Roll cost 20	100 becomes 80	20 becomes 0; 19 is refused unchanged
Recovery	50 becomes 60	95 becomes 100, not 105
Block editing	Empty cell becomes occupied	Player cell and outside coordinates rejected
Movement	One clear step succeeds	Occupied destination and world edge refused
Bridge event	One request applies once	Duplicate, malformed, and disconnected requests
Save/reload	Stored state returns	Missing, older, or damaged save handled clearly

Use a tiny isolated test world or fake endpoint first. Synthetic scenes make bugs repeatable and reduce the number of things you must investigate at once.

Debug one cause at a time

Debugging means finding why actual behavior differs from intended behavior. Reproduce the issue, record exact steps, observe the earliest error, and change one likely cause. Retest the original issue after the change.

For a game mod, compare a clean supported profile against the modified profile. Add one dependency or mod at a time. Keep the same game version and test save during comparison. Randomly changing drivers, loaders, scripts, and graphics settings together makes the result difficult to explain.

For a bridge, inspect timestamps, sequence numbers, units, axes, and pause/disconnect behavior before adjusting visual effects. For a rewrite, test the rule without rendering before blaming the engine.

Write a field note

A **field note** is a short record somebody else can use to repeat your result. It is valuable for another AI session too, because chat memory is not a reliable project archive.

Date and project revision:

OS / hardware / exact tool and game versions:

Goal and selected route:

Files changed:

Setup and run steps:

Expected result:

Observed result:

Tests passed / tests not run:

Issue: symptom → evidence → cause → fix, if known

Known limitations:

Next smallest step:

Prefer “Avatar follows the player in the test scene; collision unimplemented” to “Minecraft is fully inside Elden Ring.” The first is a useful, honest progress statement.

Package a result someone can use

Create a release folder containing the intended executable/mod, required configuration, installation guide, tested-version list, controls, limitations, credits/licenses, and removal/reset instructions. Test the package from a clean location so it does not accidentally depend on unlisted files in your development folder.

A video or screenshot can demonstrate a result, but include its version and context. Keep personally identifying desktop information, private chat, account pages, and credentials out of recordings and logs you publish.

Checkpoint: another person can follow your written run steps and understand the limits without reading your entire chat history.

18 · GitHub, public files, and GitHub Pages

What Git and GitHub each do

Git records changes in a local project. A **repository** is the project and its recorded history. A **commit** is a named checkpoint. A **branch** is a line of development. **GitHub** hosts repositories and collaboration features. **GitHub Pages** serves a website made from selected repository output.

Publishing is a separate action from saving locally. A public repository can be downloaded by other people. A Pages site is usually public too; do not assume a private development folder or chat remains private after uploading it. GitHub's publishing-source guide explains branch and workflow deployment choices. [\[S27\]](#)

A beginner publication sequence

1. Sign in to GitHub using the official website. Protect the account with a strong unique password/passkey and two-factor authentication where applicable; keep recovery information privately. [\[S28\]](#)
2. For a new project, create a repository with a clear name and description. Choose visibility deliberately. For this existing library, use the existing repository rather than creating a duplicate.
3. Prepare a release copy containing only what you intend to publish. Open its files and images before uploading.
4. Use GitHub's web upload/editor for a small change, or a Git client when you are ready for versioned local work. Review the changed-file list and the content before committing. GitHub's Hello World tutorial introduces repositories, branches, commits, and pull requests. [\[S29\]](#)
5. For a static website, make sure its entry page is `index.html` and the assets use paths that work under the repository's project URL.
6. In repository Settings → Pages, select the actual publication method: a branch/folder or the provided GitHub Actions workflow. Check the build/deployment result.
7. Open the published URL on another browser or device. Test navigation, images, PDF downloads, and shared-item links.

The library is already available at

<https://solarfren69420.github.io/infographicmegalibrary/>. Its generated static item pages provide distinct image previews, and this handbook and practice files sit under `guides/`.

Keep secrets and personal archives out

Do not upload passwords, API tokens, private SSH keys, login cookies, recovery codes, billing documents, full game installations, or unreviewed raw chat exports. Check text documents, screenshots, PDFs, and logs as well as source code. A preview image can accidentally show a secret just as a configuration file can.

For local Git projects, `.gitignore` helps exclude intended private/generated files. It does not retroactively remove a tracked file or its old commits. Before a commit, review exactly which files are staged. Prefer selecting the intended files rather than uploading the entire Downloads folder.

GitHub push protection can block supported secret patterns before publication. A scan is useful, but cannot establish that every credential, personal detail, or security flaw is absent. Review findings and keep dependencies maintained. [S30]

If a real credential is exposed, revoke or rotate it at its provider first. Deleting the visible file does not invalidate the credential or erase previous copies. History cleanup can require coordination and should follow GitHub's instructions rather than an improvised force push. [S31]

Websites cannot hide a browser-side secret

Anything shipped to a public browser—including HTML, JavaScript, configuration, and downloads—is available to the visitor. A static Pages site cannot safely keep a private API key in its JavaScript. If a future feature needs a secret, use an appropriately secured server-side service instead of embedding that secret in the gallery.

The current library browses static files without asking visitors for accounts or private tokens. Local raw chat and the original backup are separate from intended site output.

Sharing an infographic or the handbook

In the image viewer, use **Share page** rather than only copying a gallery fragment. On an item page, use **Copy share link**. A shared item's page contains its own title and image metadata. For the handbook, share its PDF link or the library link with the download button.

An X or messaging preview may be cached or omitted by the platform. That is distinct from whether the link opens correctly. If the site works but an old card remains, allow for the platform's cache behavior rather than renaming your whole collection.

Checkpoint: the release contains only intended public material, and the live site opens its images and document links correctly. Account security and running-server security require their own checks beyond scanning a repository.

19 · Subscriptions, student offers, and payment claims

Learn first; buy for a specific need

The practical exercises in this handbook do not need a paid AI plan. Start with the tools and access you already have. If you later choose a subscription, identify the project it serves, the actual price and tax, the billing provider, renewal date, and cancellation procedure.

A chat subscription and API usage billing can be separate products. Do not assume paying for one gives unlimited use of the other. Check the provider's current plan/billing page for the interface you intend to use, and avoid creating duplicate subscriptions through the web and a phone's app store.

The four-month student offer

The four-month image corresponds to a real official offer as checked on 3 October 2026. The official page says eligible current full-time or part-time students at qualifying U.S. degree-granting institutions can claim four free monthly periods of ChatGPT Plus, including Work, by **31 October 2026**. Enrollment is verified through SheerID. This is a time-limited snapshot, not a permanent benefit. [\[S32\]](#)

Use the official offer page, sign in to the intended ChatGPT account, complete verification, and return to finish activation. The official terms say a payment method is required; the plan then renews at \$20/month unless canceled. Existing directly billed monthly Plus users may qualify; an active Apple/Google store-billed subscription is not eligible for application while it remains store-billed. Canceling early forfeits remaining promotional months after the current billing period. Recheck the live terms before acting. [\[S33\]](#)

The altered “47 months free” image was identified as spam and is not a real offer. Do not enter account or school credentials into a page merely because a poster or message links to it.

Pay-in-four and Google Play: what the image proposes

The archived payment poster suggests an installment service → a Google Play gift card → an Android in-app subscription. Treat that as a conditional idea, not a confirmed universal checkout method. Approval, gift-card restrictions, country/currency, supported subscriptions, merchant acceptance, and billing rules can block parts of the chain.

Do not buy a gift card assuming it will fund a particular subscription until the actual merchant, payment provider, and subscription checkout confirm it. Gift cards and installment debt are not the same as a refundable monthly trial. This handbook has not verified the proposed chain across all named providers or

accounts.

A smaller first payment does not reduce the total

A hypothetical \$20 purchase split evenly across four payments still totals \$20 before any fees. Several small plans can overlap. The CFPB explains that missed BNPL payments can produce late fees, account restrictions, debt collection, and possible credit-report consequences. Read the actual agreement and due dates. [\[S34\]](#)

The library's warning poster makes the useful point that an AI subscription does not guarantee future earnings. Choose a plan only when its total cost fits your budget and it has a specific purpose. Set renewal reminders and cancel through the provider that bills you if you no longer need it. Keep payment and verification documents outside public projects.

Checkpoint: you know the total obligation, renewal date, and billing provider before subscribing. If any part of a promoted payment route is uncertain, do not treat the infographic as checkout confirmation.

20 · Troubleshooting without making the problem bigger

Capture useful context

Write down your OS/version, tool version, current folder, command or action, first meaningful error, and what you expected. If sharing a log, remove account tokens and personal information first. Include relevant error text rather than an unreadable full-screen photo when possible.

Symptom	Likely thing to check	Small next action
File opens as text instead of an app	Wrong file type or source file	Check extension; compile source with its proper tool
command not found / not recognized	Tool missing or not on PATH	Check install completed; reopen terminal; use official setup help
Program cannot find a file	Wrong current folder or path	Print location and list files; quote the actual path
JSON parse error	Commas, quotes, bracket structure	Compare against the complete example; validate before loading
Rust linker error	OS compiler/linker prerequisite	Follow the Windows/macOS/Linux toolchain instructions
Ghidra asks for Java or fails at launch	JDK mismatch or location	Read the selected release's JDK requirement and launch log
Minecraft mod refuses to load	Game/loader/API/Java mismatch	Compare exact versions in the mod's requirements
Bridge will not connect	Server stopped, wrong transport/port	Test documented local endpoint and inspect both logs
Visual bridge is offset or flickering	Axes, scale, projection, timing	Reproduce with a fixed test scene and recorded camera

Symptom	Likely thing to check	Small next action
Overlay objects have no collision	Only rendering was implemented	Add a defined physics/collision route; test one object
Works on one computer only	Unlisted dependency or OS-specific code	Test a clean package and state tested platforms
Pages link gives 404	Deployment, path, or filename case	Check Actions/Pages status and exact published URL

OS-specific reminders

Windows: PowerShell and CMD interpret some commands differently. Use the matching instructions. Check that a file is not secretly named `main.rs.txt`. If a downloaded project is a ZIP, extract it first. A Windows `.exe` does not become a macOS/Linux program by removing the extension.

macOS: use plain-text mode for code edits. An Intel build and an Apple Silicon build may have different requirements. If a downloaded tool triggers a security or compatibility message, follow that publisher's current instructions for your version; do not disable OS protections globally to force it to run.

Linux: distribution names matter for package commands. A Debian `.deb` guide is not an Arch or Fedora installation recipe. File names may be case-sensitive. A script may need the documented launch method and execution permission, but do not change permissions across your entire home folder. Wayland and X11 can differ for capture or global shortcut features.

PATH, ports, and versions in plain language

PATH is the list of locations where a shell looks for commands. Installing a tool and refreshing the shell's environment often makes it available. Randomly copying binaries into system folders is not a good first fix.

A **port** identifies a network service endpoint. “Address already in use” often means another process already occupies that port. Identify that process or choose another documented local port; opening a router port is not the solution to a local address conflict.

Version mismatch means the pieces expect different interfaces. Pin a known compatible set and record it. Updating every component at once may introduce a new mismatch instead of solving the old one.

A short request for debugging help

OS and version:

Tool/game/loader versions:

Current project folder:

Command or exact action:

First error:

Expected result:

What worked before this:

Please identify the first likely cause, explain it simply,
and give one small check before suggesting any system changes.

Checkpoint: you can reproduce the problem and name the next check. If you cannot, simplify the test instead of adding more tools.

21 · Copy-paste prompts and a practical next step

Use the image as a frame of reference

The owner's intended use is a quick brainstorm or one-shot prompt for an AI. Attach the relevant infographic, then add your actual goal and environment. The picture can convey the idea faster than a long description, but the assistant should still inspect the real project and verify assumptions.

The following templates consolidate the repeated poster prompts into reusable starting points. They do not promise a whole mashup in one response. Replace bracketed fields and keep the request as small as your current milestone.

Prompt A: route selection for a complete beginner

Use this infographic as brainstorming reference, not verified facts.
I am a beginner using [OS/version]. My goal is [one feature].
The host is [game/app and version]. The other reference is [name].
First inspect the available project and official modding options.
Compare native recreation, content port, bridge, and rewrite.
Do not assume Rust or two running games are required.
Choose the simplest supported route and explain why in plain language.
Create MODDING_PLAN.md with prerequisites, exact versions,
one vertical slice, affected files, unknowns, and a test checklist.
Separate observed facts from assumptions and unverified concepts.

Prompt B: build one original mechanic

Implement one original prototype mechanic from this specification:
[paste the short rule/data record].
My chosen language/engine and exact version are [details].
Use original placeholders. Explain where every created file goes.
Build the smallest runnable example and check its boundary cases.
Report commands run, results, and tests you could not perform.
Give beginner run/reset steps and a brief known-limitations section.
Do not add unrelated mechanics, multiplayer, or paid services.

Prompt C: investigate a passthrough

I want to investigate a local single-player bridge between
[host/version] and [companion/version] on [OS].
Use the attached picture only as an initial architecture sketch.
Identify supported integration points and current dependencies.
Plan fake endpoints first, then one state transfer, then one event.
Define units, authority, event IDs, timing, disconnect, and pause.
Treat rendering/depth/collision as separate optional milestones.
Build only the first approved slice and verify it where possible.
Do not bypass anti-cheat or distribute commercial game files.
Document actual results and unknowns for the next session.

Prompt D: BONELAB-native first slice

Use my Project Third Eye/BONELAB idea as a behavior reference.
First inspect exact game versions and current Unity/Marrow support.
Goal: one original tool, one simple car, one BONELAB test level.
Prefer native recreation if the supported tools can do the job.
Do not assume Rust, a full rewrite, or a live passthrough is needed.
Explain what the SDK supports and what requires another mod route.
Create a small plan, then implement the simplest working component.
Give VR input, grabbing, collision, vehicle, reset, and performance checks. Clearly mark in-game tests that I must perform myself.

Prompt E: organize research and continue later

Summarize this session into a project field note and mechanic records.
Include exact versions, source paths, evidence, uncertainty,
files changed, tests passed, failures, and the next smallest step.
Avoid copying repeated discussion into every record.
Do not put passwords, tokens, personal chat, or billing data in output.
Make the note useful to a new assistant without this chat history.

A sequence you can finish

First session: make the practice folder, open the browser exercise, test its six rules, and save your observations. **Next session:** choose one actual project route and complete its version inventory. **Then:** build one working slice, record the evidence, and package it only after a clean install/run check.

There is no fixed completion-time promise. Your speed depends on experience, tooling, compatibility, and project scope. Progress means the next verified milestone, not how many impressive images or generated files you accumulate.

Checkpoint: you have a usable prompt, a small task, and a clear success condition. Keep the other ideas in a future-work list.

A · Plain-language glossary

Term	Meaning in this handbook
API	A documented way one piece of software asks another to do something
Architecture	The organization of systems; also a CPU family such as x64 or arm64
Asset	Content such as a texture, sound, model, or animation
Backup	A recoverable copy kept before something is changed or lost
Binary	Compiled program/data, rather than human-readable source instructions
Branch	A line of development in a Git repository
Build	Turn source and resources into a runnable program or package
CLI	Command-line interface: using text commands
Client	The user-side app, possibly communicating with a server
Collision	Rules determining which objects block, touch, or hit others
Commit	A recorded Git checkpoint
Compiler	A tool that translates source code into another executable form
Component	One category of data attached to an entity
Configuration	Adjustable settings separated from the main code
Crate	A Rust compilation/package building block
Database	Structured records that can be stored, related, and queried
Debug build	A development-oriented build that may retain extra diagnostic clues
Debugging	Finding the cause of behavior that differs from expectations

Term	Meaning in this handbook
Decompilation	Attempting to express compiled behavior in higher-level form
Dependency	A tool, library, asset, or system another part needs
Depth buffer	Rendering data used to determine distance/occlusion
Deployment	Publishing built output to a place where others can use it
Disassembly	Translating machine instructions into assembly representation
Donor	A source of permitted content or inspiration for a host project
ECS	Entity-component-system: organizing objects, their data, and update rules
Engine	Reusable software for running/rendering game or app behavior
Entity	An object or thing tracked by a game
Extension	The filename ending indicating a type, such as .pdf or .rs
FFI	An interface for calling code written in another language
Field note	A concise, repeatable record of setup, results, and limitations
Frame	One displayed image in an animation or rendered app
GUI	Graphical user interface: windows, buttons, and menus
Hash	A fingerprint used to compare file contents
Host	The game/app where the user experiences an integrated feature
Idempotent	Repeating the action leaves the same intended final result
IPC	Communication between separate running processes
JSON	A structured text format for storing/exchanging values
JDK	Java Development Kit: tools/runtime needed for Java development
Library	Reusable code; here also the collection of infographics

Term	Meaning in this handbook
Linker	A tool that combines compiled parts and required libraries
Loader	Software that loads the mods/plugins appropriate to a game
Localhost	The current computer as a network destination
MCP	A standard for connecting AI applications with tools and data
Mod	A modification or extension of an existing game
Module	A group of related code or a well-defined subsystem
OS	Operating system: software managing apps, files, and hardware
Ownership	Rust's rules about responsibility for values and their lifetime
Parity	Agreement with a defined reference or expected inventory
Passthrough	A design that bridges live processes rather than rewriting them
PATH	Locations where a shell searches for executable commands
Port	A numbered service endpoint on a network address
Process	One running instance of a program
Prompt	A request to an AI; also a terminal's ready-for-input indicator
Prototype	A small experimental implementation used to test an idea
Query	A request for information, often from a database
Repository	A project and its versioned history
Reprojection	Adjusting rendered output using a changed camera pose
Reverse engineering	Studying an existing system to understand its operation
Runtime	The running program and the environment supporting it
SDK	Software Development Kit: tools/interfaces for a platform
Serialization	Turning structured values into a storage/transmission format

Term	Meaning in this handbook
Shell	The command interpreter inside a terminal
Source code	Editable instructions written in a programming language
Stamina	An example resource spent by actions and restored by recovery
Symbol	A named item or location in a program
Terminal	A window used to enter and see text commands
Toolchain	The cooperating tools used to build software
Vertical slice	One small feature working through input, logic, and output
Voxel	A volume cell, often represented visually as a block
Xref	Cross-reference showing a use/link between program locations

B · Coverage of the original collection

What was included and consolidated

The catalog contains 49 downloaded attachments: 32 infographics, three supporting references, one community text prompt, two owner notices, eight spam items, and three exact repeated uploads. That is 46 distinct file hashes. The original chat records 52 attachment references; three opening images did not survive the export's repeated-filename overwrite. This handbook does not invent their contents.

The original infographics remain unchanged in the library. The following index is a coverage map, not a claim that every original technical statement is correct. Original title links open the item's share page so you can inspect the picture at full size.

item-01 · Combining Game Clients with Rust

Chapters 5, 8–9, 14–15 · Binding, modding, and reimplementing routes.

item-02 · Why Rust Feels Supernatural

Chapters 8–10, 14 · Consolidated Rust and modular data-driven design.

item-03 · The Database Method

Chapters 10–12, 14 · Evidence records, indexing, parity, and implementation.

item-05 · CHASM Server Overview

Chapters 11, 15, 17 · Archived community snapshot and project verification.

item-06 · decomp.dev Overview

Chapters 11 · Matching reconstruction versus portable rewrites; live percentages unverified.

item-07 · Why the Bully Debug Build Matters

Chapters 11 · Debug artifacts as clues, with documented Bully example.

item-09 · GTA 6 as a Systems Goldmine

Chapters 11, 16 · Open-world systems as inspiration; extraction not established.

item-10 · Combining Games with Rust and AI — Starter Path

Chapters 7–10, 14–15 · Small combat/building prototype before larger systems.

item-11 · AI Decompilation and Rust — Mashup Starter Path

Chapters 6, 10–12, 14 · AI-assisted stages; garbled automation commands not reused.

item-12 · Can I Combine Elden Ring and Minecraft

Chapters 5, 7, 15 · Original combat/building rules rather than merging installations.

item-13 · Minecraft Inside GTA V — Passthrough Mod

Chapters 13, 15 · GTA/Minecraft state, events, depth, timing, and platform constraints.

item-16 · Minecraft Inside Elden Ring — Passthrough Concept

Chapters 13, 15 · Elden Ring bridge architecture remains a conceptual adaptation.

item-17 · Minecraft Character Inside Elden Ring

Chapters 13, 15, 21 · Avatar/HUD first slice and explicit integration limits.

item-19 · Any Game × Any Game × Any Game

Chapters 9, 16 · Creative roster organized into compatible subsystem ideas.

item-20 · The Full Pipeline

Chapters 8–14, 17 · Research → records → modules → tested runtime.

item-21 · The Mega Game Combo Engine

Chapters 16 · Genre categories, constraints, and corrected subset counts.

item-22 · Metal Gear Rising — Absurd Fusion

Chapters 16 · High-speed combat/world fusion as an inspiration menu.

item-23 · What the Hell Is This Combo

Chapters 16 · Many-game roster narrowed to small experiments.

item-24 · GMod × Minecraft — Voxel Sandbox Fusion

Chapters 5, 15–16 · Native voxel sandbox first; physics/circuits later.

item-30 · Passthrough Mod vs Rust Rewrite Client

Chapters 5, 13–14 · Two live processes versus one newly implemented runtime.

item-31 · Passthrough Mod — First Prompt for the AI

Chapters 13, 21 · Cleaned bridge prompt with staged verification.

item-32 · Rust Rewrite Client — First Prompt for the AI

Chapters 10, 14, 21 · Cleaned rewrite prompt with specifications and tests.

item-33 · Claude and ChatGPT — Install Guide

Chapters 2, 6, 20 · Official downloads by OS; desktop and CLI distinguished.

item-34 · Real Commands to Open Them

Chapters 4, 6, 20 · Current tool entry points; archived flags not installation instructions.

item-36 · Ghidra and Gemini CLI — Setup Guide

Chapters 11–12 · Ghidra/Gemini prerequisites, MCP, and compatible version sets.

item-37 · Pay-in-4 to Google Play — AI Subscription Guide

Chapters 19 · Proposed payment chain; checkout/account support not verified.

item-38 · Students — Four Months Free Offer

Chapters 19 · Real four-month offer checked; dates and billing terms stated.

item-42 · Do Not Finance an AI Subscription Lightly

Chapters 19 · Total obligation, renewal, and BNPL risk consolidated.

item-43 · Postal 2 Inside Minecraft

Chapters 5, 15, 21 · Postal/Minecraft proposal and SkyCraft compatibility limits.

item-44 · What Even Is Rust

Chapters 8–9 · Rust basics, compiler, Cargo, and subsystem boundaries.

item-47 · Why Rust Dominates This Pipeline

Chapters 8–9, 14 · Rust strengths and realistic engineering tradeoffs.

item-48 · Why Rust Is Perfect for This Pipeline

Chapters 8–9, 14, 20 · Memory-safety and cross-target claims corrected.

Supporting discussion and exclusions

The “What You Are Really Doing” screenshot is covered by chapters 8–11; the combination-math note is corrected and explained in chapter 16; the Minecraft/Elden Ring gameplay reference informs the visual-bridge discussion without proving the conceptual adaptation is implemented. The human Postal 2 prompt is preserved and explained in chapter 15, with cleaner reusable prompts in chapter 21.

The BONELAB/Project Third Eye question and its native-first prompt come from the chat and are covered in chapters 5, 15, and 21. The owner's concept-art and high-zoom notices are reflected in chapter 1. Repeated posts are consolidated, and spam/joke images are not treated as factual instructions.

Corrections and limitations that matter

The handbook treats the images as one-shot ideas, validates actual exercises separately, and does not reproduce garbled code as runnable instructions. It separates GUI apps from coding CLIs, defers installation to the correct current OS/release instructions, checks the real student offer, and labels the payment chain as unverified.

Rust is useful but optional for many native mods. Memory safety is not a promise of zero vulnerabilities or leaks. Cross-platform source still needs compatible dependencies, build tools, and testing. A passthrough picture is not a shared physics simulation. A database organizes evidence rather than replacing it. The combination totals are calculated explicitly, with assumptions stated.

No complete commercial game rewrite, universal mod installer, automatic decompilation workflow, or working new cross-game bridge is included. The supplied browser playground, Rust console exercise, and SQLite learning script are original teaching artifacts with a smaller, testable scope.

C · Sources, verification, and exercise files

How the sources are used

Bracketed references such as [S13] link to the entries below. The practical source links were checked on 3 October 2026, except where the guide explicitly reports an unavailable live page. Publisher documentation is used for current tool behavior; the supplied collection and chat provide the original ideas and topic coverage. The glossary, project planning advice, examples, and prompt templates are written as this handbook's explanatory material.

Source pages may change. When installing software, use the current documentation for the exact release rather than copying a version-specific command from an old image. A verified documentation statement is not a claim that every installation was performed on Windows, macOS, and Linux during preparation.

S01 · Microsoft · File Explorer in Windows

<https://support.microsoft.com/en-gb/windows/experience/fileexplorer/file-explorer-in-windows>

S02 · Microsoft · Common filename extensions

<https://support.microsoft.com/en-us/windows/experience/storage-filemanagement/common-file-name-extensions-in-windows>

S03 · Apple · Open or quit Terminal

<https://support.apple.com/guide/terminal/open-or-quit-terminal-apd5265185d-f365-44cb-8b09-71a064a42125/mac>

S04 · OpenAI · ChatGPT quickstart

<https://learn.chatgpt.com/docs/quickstart>

S05 · OpenAI · ChatGPT desktop app for Linux

<https://learn.chatgpt.com/docs/linux/linux-app>

S06 · Anthropic · Claude downloads and platform information

<https://claude.com/download>

S07 · OpenAI · Codex CLI

<https://learn.chatgpt.com/docs/codex/cli>

S08 · Anthropic · Claude Code quickstart

<https://code.claude.com/docs/en/quickstart>

S09 · Google · Gemini CLI installation

<https://geminicli.com/docs/get-started/installation/>

S10 · Node.js · Official downloads

<https://nodejs.org/en/download>

S11 · Godot · Your first 2D game

https://docs.godotengine.org/en/stable/getting_started/first_2d_game/index.html

S12 · Rust Book · Ownership

<https://doc.rust-lang.org/book/ch04-01-what-is-ownership.html>

S13 · Rust Book · Installation

<https://doc.rust-lang.org/book/ch01-01-installation.html>

S14 · Fabric · Development environment setup

<https://docs.fabricmc.net/develop/getting-started/setting-up>

S15 · Facepunch · Garry's Mod Lua programming introduction

https://wiki.facepunch.com/gmod/Beginner_Tutorial_Intro

S16 · Stress Level Zero · MarrowSDK documentation

<https://github.com/StressLevelZero/MarrowSDK/blob/main/Docs/Home.md>

S17 · Rust Book · Hello, Cargo

<https://doc.rust-lang.org/book/ch01-03-hello-cargo.html>

S18 · Rust Book · Writing tests

<https://doc.rust-lang.org/book/ch11-01-writing-tests.html>

S19 · Bevy · Quickstart introduction

<https://bevy.org/learn/quick-start/introduction/>

S20 · SQLite · Quickstart and browser Fiddle link

<https://www.sqlite.org/quickstart.html>

S21 · SQLite · Command-line shell

<https://www.sqlite.org/cli.html>

S22 · NSA Ghidra · Getting Started (release-specific requirements)

<https://github.com/NationalSecurityAgency/ghidra/blob/master/GhidraDocs/GettingStarted.md>

S23 · Hidden Palace · Bully debug-build documentation

[https://hiddenpalace.org/Bully:_Scholarship_Edition_\(Debug_Build\)](https://hiddenpalace.org/Bully:_Scholarship_Edition_(Debug_Build))

S24 · Model Context Protocol · Introduction

<https://modelcontextprotocol.io/docs/getting-started/intro>

S25 · GhidraPluginProject · Third-party Ghidra MCP bridge

<https://github.com/GhidraPluginProject/ghidra-mcp>

S26 · chasmlol · SkyCraft requirements and installation

<https://github.com/chasmlol/SkyCraft>

S27 · GitHub · Configure a GitHub Pages publishing source

<https://docs.github.com/en/pages/getting-started-with-github-pages/configuring-a-publishing-source-for-your-github-pages-site>

S28 · GitHub · Two-factor authentication

<https://docs.github.com/en/authentication/securing-your-account-with-two-factor-authentication-2fa/about-two-factor-authentication>

S29 · GitHub · Hello World repository tutorial

<https://docs.github.com/en/get-started/using-github/hello-world>

S30 · GitHub · Push protection

<https://docs.github.com/en/code-security/concepts/secret-security/push-protection>

S31 · GitHub · Removing sensitive data from a repository

<https://docs.github.com/en/authentication/keeping-your-account-and-data-secure/removing-sensitive-data-from-a-repository>

S32 · OpenAI · 2026 student offer claim page

<https://chatgpt.com/students/2026/>

S33 · OpenAI · Back to School offer terms

<https://help.openai.com/en/articles/20001493-chatgpt-back-to-school-offer-for-students>

S34 · CFPB · Consequences of unpaid BNPL loans

<https://www.consumerfinance.gov/ask-cfpb/what-happens-if-i-cant-pay-back-a-buy-now-pay-later-bnpl-loan-en-2116/>

Companion files

The [browser playground](#) is a complete self-contained exercise. The [Rust source](#), [JSON mechanic record](#), and [SQLite script](#) let you avoid retyping the longer examples. Save source files into the locations explained in chapters 8 and 10; they do not all run by double-clicking.

The editable [handbook source](#) and PDF generator are kept with the library for future revisions. Original images, source metadata, and separate archive collections remain available there too.

Edition: SolarFren · Infographic Mega Library Beginner Handbook · 3 October 2026. Prepared from the retained library and discussion, with original exercises and checked primary-source references. The aim is a usable first step, then a clear path from an idea to an honestly tested result.